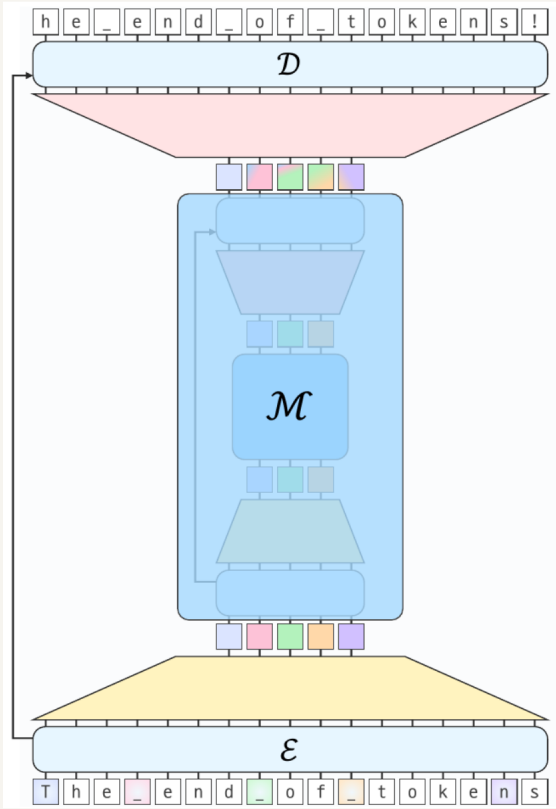
PAPER DISCUSSION · ICLR 2026

# Dynamic Chunking for End-to-End Hierarchical Sequence Modeling

**H-Net** · Sukjun Hwang, Brandon Wang, Albert Gu
Carnegie Mellon University & Cartesia AI

Presented by Burak Suyunu · LifeLu Reading Group · 09.10.2026




Paper Fig. 1 (left)

0 · OPENING

# Why this paper, for a protein group?

TEXT · BYTES

### Learns its own tokenizer

Beats a BPE Transformer at matched data and compute (this paper).

→

DNA · NUCLEOTIDES

### Already adapted 3×

3.6× data efficiency in the paper; DNAChunker, LDARNet, dnaHNet at ICML 2026.

→

PROTEINS · AMINO ACIDS

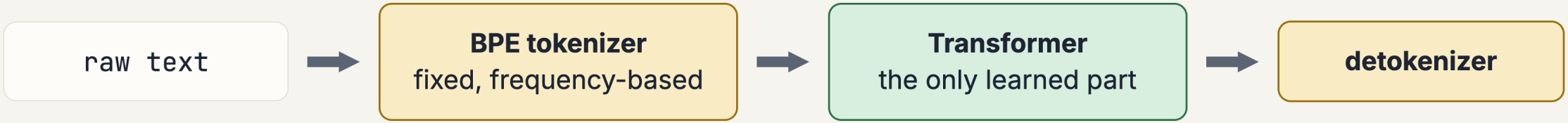
### Not yet adapted

The opening for us: can a protein model learn its own units?

**1** The problem | **2** Mamba primer | **3** Architecture | **4** Experiments | **5** Toward proteins

> Protein language models inherit their input units from NLP: one token per residue, chosen by hand. H-Net asks whether the model can learn the right units itself. By the end you should be able to judge whether that makes sense for proteins.

# Tokenization is a hand-made preprocessing step




### Spelling

“How many r's in strawberry?” The model sees a few subword tokens, never the individual letters.

### Other languages

Vocabularies built mostly on English handle Chinese, code or DNA poorly: no spaces, weak boundaries.

### Noise

**HeLLo** and **hello** become completely different token sequences.

> Every modern LLM compresses text with a fixed tokenizer before the network sees it. That compression is not learned together with the model, and it causes known failures in spelling, non-English languages, and noisy text.

# Bytes vs. characters

❓ **Question I had:** what is the difference between a byte-level and a character-level model?

| Char | Unicode | UTF-8 bytes | Bytes |
|---|---|---|---|
| a | U+0061 | 61 | 1 |
| é | U+00E9 | C3 A9 | 2 |
| 中 | U+4E2D | E4 B8 AD | 3 |
| 😀 | U+1F600 | F0 9F 98 80 | 4 |

### Character-level

Predicts 中 in 1 step. Vocabulary ≈ 150k possible characters: rare ones are missing or bloat the table.

### Byte-level (H-Net)

Predicts E4 → B8 → AD in 3 steps. Vocabulary = 256, covers any text, code or data.

For English the two are the same, because ASCII letters are one byte each. Bytes give a tiny fixed vocabulary that covers anything, at the price of longer sequences: Chinese becomes 3× longer, and a lone byte like E4 means almost nothing.

1 · THE PROBLEM

# Why tokenizers survive, and what has been tried

1 · ISOTROPIC

### No compression

MambaByte, ByT5. Every layer sees every byte: very expensive.

2 · STATIC

### Every k bytes

Hourglass, MegaByte. Pooling splits words at arbitrary places.

3 · EXTERNAL

### Hand-made rules

Spaces (SpaceByte) or entropy from a separate model (BLT). Not learned, can't be nested.

4 · LEARNED

### End-to-end

DPT (Gumbel noise): unstable, one stage. **H-Net**: stable and nestable.

❓ **Question I had: what does "isotropic" mean?** The same block repeated at constant length and width. A BPE Transformer is isotropic too: its hierarchy is hidden in the tokenizer, outside the network.

Tokenizers compress text about 4.6× for zero FLOPs, a very cheap compute multiplier. Earlier tokenizer-free models either did not compress, compressed blindly, relied on hand-made rules, or were unstable when learned (paper Table 4).

# Transformers remember like a database

KV CACHE AFTER 8 TOKENS · ONE ENTRY PER TOKEN, KEPT FOREVER




The new token's query is compared against every stored key, then mixes the matching values.

**Memory**
grows with every token

**Training cost**
quadratic in length, $O(L^2)$

**Strength**
exact recall of any past token

> Attention keeps a key and a value for every past token and compares each new token against all of them. That gives perfect recall, but cost grows with length: quadratic in training, and a cache that keeps growing at inference.

# Mamba remembers like a running summary

❓ **Question I had:** what is Mamba, if I only know Transformers?

STATE SPACE MODEL · ONE STEP PER TOKEN

$$h_t = A_t \cdot h_{t-1} + B_t \cdot x_t$$

$$y_t = C_t \cdot h_t$$

**$h_t$** fixed-size state: the only memory
**$A_t$** how much of the old state to keep
**$B_t$** how much of the new input to write
**$C_t$** what to read out

READING “prod” BYTE BY BYTE

p → state ≈ “…p”
r → state ≈ “…pr”
o → state ≈ “…pro”
d → state ≈ “…prod”

Same-size state at every step; older details fade unless worth keeping.

Mamba is a state space model: one fixed-size state, updated once per token. Cost is linear in length and constant per step, but anything not written into the state is lost. Gu's analogy: Transformers are databases, SSMs are brains.

# Selectivity, Mamba-2, and the trade-off

**Selective:** $A_t$, $B_t$, $C_t$ are computed from the current input, so the model decides per token what to keep and what to forget.

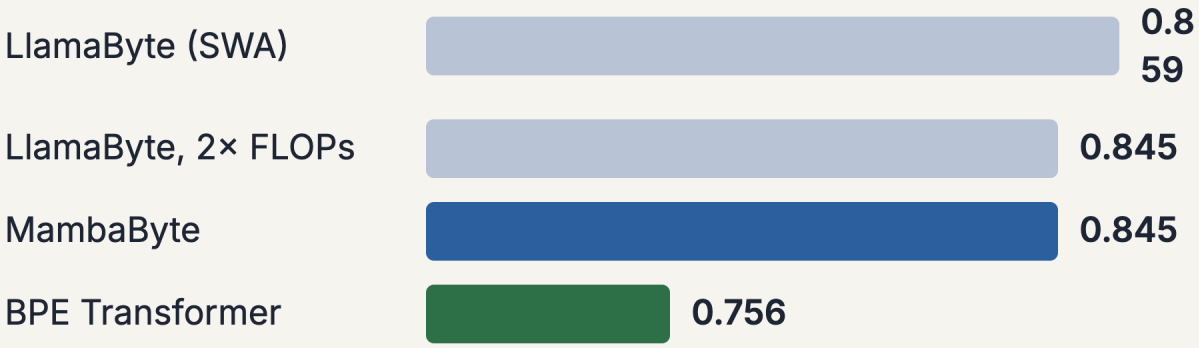
**Mamba-2:** simpler A (one decay per head), so training runs as fast matrix multiplications; close to linear attention.

| | Transformer layer | Mamba-2 layer |
|---|---|---|
| Memory | every past token (KV cache) | fixed-size compressed state |
| Cost vs. L | quadratic; growing per step | linear; constant per step |
| Strength | precise retrieval | compressing a stream |
| Weakness | expensive on long sequences | lossy: no exact recall |
| Params | ≈ $12 \cdot D^2$ (attention $4D^2$ + MLP $8D^2$) | ≈ $6 \cdot D^2$ (no MLP in H-Net's outer layers) |

> Selectivity is what lets Mamba handle language: in principle it can keep accumulating inside a word and refresh at a boundary (an illustration, not something the paper measures). The parameter counts explain the ablations later: one Transformer layer ≈ two Mamba-2 layers.

# Attention needs meaningful tokens

FINAL VALIDATION BPB · LARGE SCALE · LOWER IS BETTER (axis starts at 0.70)




Source: paper Table 2 (all byte models on raw bytes, BPE on GPT-2 tokens).

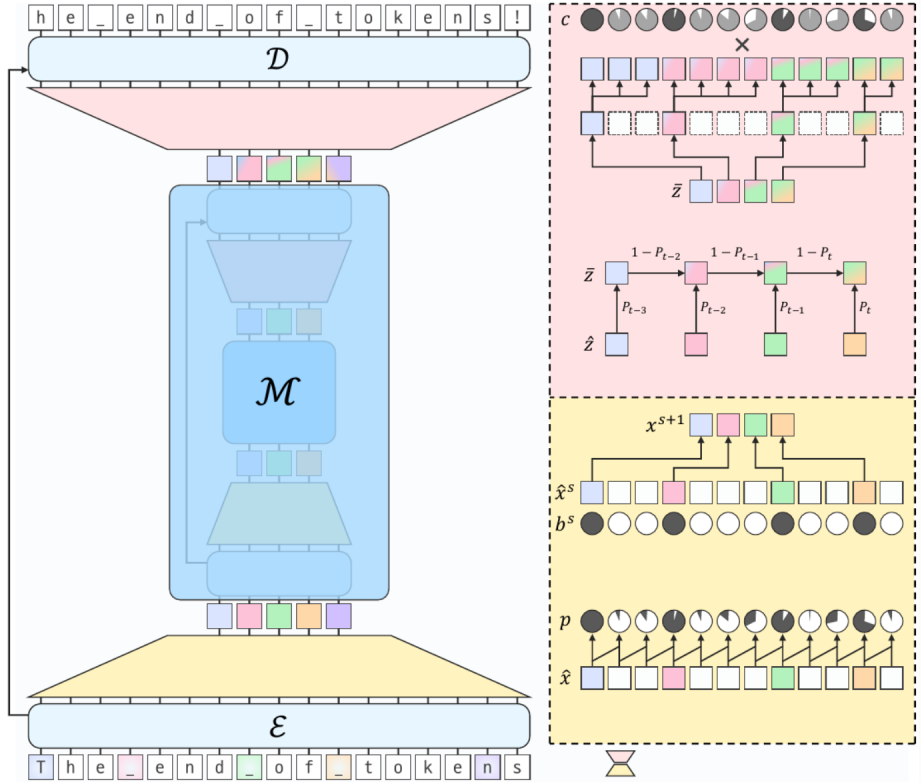
GU'S THESIS

**Attention works best on data already compressed to the right level of abstraction.**

A single byte means little on its own; compressing it into units is what SSMs do naturally.

> On raw bytes, Mamba matches a Transformer that gets twice the compute and beats the compute-matched one, while all of them trail the tokenized model. This motivates the whole design: Mamba at byte resolution, attention on meaningful chunks.

# The big picture: a U-Net for sequences




```
xˢ = Eˢ(xˢ) encode stage s
(xˢ⁺¹, pˢ) = Chunk(xˢ) pick boundaries
ẑ = M(x) main net, shortest seq.
zˢ = Dechunk(ẑˢ⁺¹, pˢ) + Linear(xˢ)
ẑˢ = Dˢ(zˢ) decode stage s
```

Encoder / decoder | Chunking | Main network | Dechunking

Left: encoder compresses, main network thinks, decoder expands; two stages here (S = 2). Right: the dynamic chunking modules. Yellow = chunking: (a) routing, (b) downsampler. Pink = dechunking: (c) smoothing, (d) upsampler. Each returns on its own slide.

# Meet the last 8 bytes of “…new product!”

| p | r | o | d | u | c | t | ! |
|---|---|---|---|---|---|---|---|
| 0x70 · t=1 | 0x72 · t=2 | 0x6F · t=3 | 0x64 · t=4 | 0x75 · t=5 | 0x63 · t=6 | 0x74 · t=7 | 0x21 · t=8 |




**The story:** early in training, the router finds the right cut **pro|duct**, but also makes a hesitant extra cut at **c**. We watch every component handle this bad boundary, and training remove it.

> Vectors are replaced by small scalar numbers so the arithmetic stays visible. Morphologically the ideal split is pro|duct; the paper's Figure 4 uses the same word.

# ① Encoder: give every byte context

RUNNING EXAMPLE · ①

| p | r | o | d | u | c | t | ! |
|---|---|---|---|---|---|---|---|

❓ **Question I had:** what do the encoder, the main network and the decoder each actually do?

MAMBA STATE AFTER EACH BYTE · 4 MAMBA-2 LAYERS · WIDTH 1,024 · ALL 8,192 BYTES

| …_p | …pr | …pro | …prod | …rodu | …oduc | …duct | …uct! |
|---|---|---|---|---|---|---|---|

**Job 1 · feed the router**
make “a new unit starts here” detectable

**Job 2 · pack the kept vectors**
each chunk's content must fit in one vector

**Job 3 · feed the decoder**
fine detail via the residual Linear(x̂)

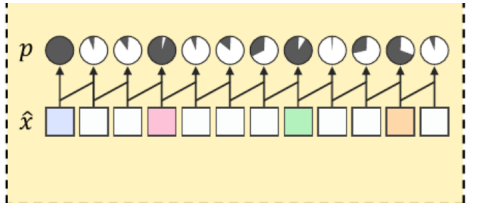
Because it is Mamba, each output is a compressed digest of everything read so far. Note: a kept vector sits at a chunk's **first** byte, so the vector at d already summarizes “...pro” plus d.

# ② Routing module: compare neighbours

RUNNING EXAMPLE · ②

p r o d u c t !

$$q_t = W_q \cdot \hat{x}_t \quad k_t = W_k \cdot \hat{x}_t$$
$$p_t = \tfrac{1}{2} \cdot (1 - \cos(q_t, k_{t-1})) \in [0, 1]$$
$$b_t = 1 \text{ if } p_t \geq 0.5$$



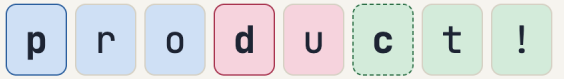

Paper Fig. 1(a) · routing module

| t | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 |
|---|---|---|---|---|---|---|---|---|
| byte | p | r | o | d | u | c | t | ! |
| $\cos(q_t, k_{t-1})$ | -1.0 | 0.6 | 0.8 | -0.8 | 0.4 | 0.0 | 0.9 | 0.2 |
| $p_t$ | 1.0 | 0.2 | 0.1 | 0.9 | 0.3 | 0.5 | 0.05 | 0.4 |
| $b_t$ | 1 | 0 | 0 | 1 | 0 | 1 | 0 | 0 |

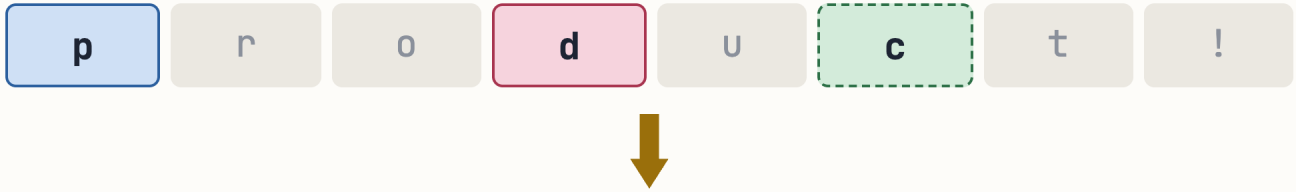
> Dissimilar neighbours mean “new unit”: confident at d (0.9), hesitant at c (0.5). In the figure, each pie is one $p_t$, computed from $\hat{x}_t$ and $\hat{x}_{t-1}$. p gets 1.0 after a space; at sequence start $p_1 = 1$ by definition.

# ③ Downsampler: keep the boundary vectors

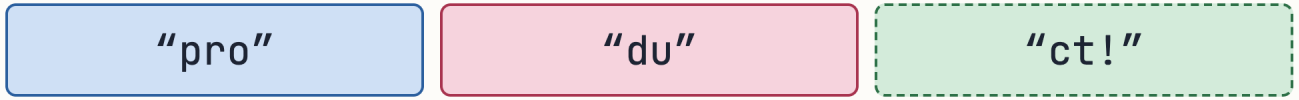
RUNNING EXAMPLE · ③

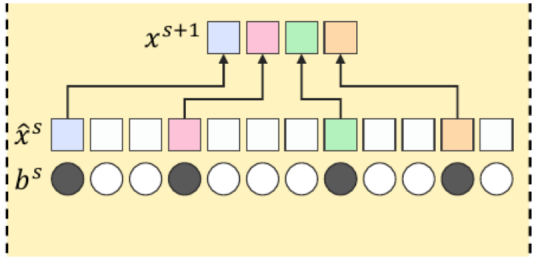



ENCODER OUTPUTS x^(8 VECTORS)

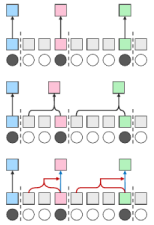


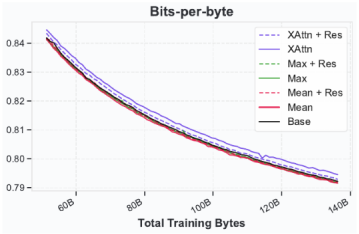

x¹: 3 CHUNK VECTORS · P = [1.0, 0.9, 0.5] · F = 3/8




Paper Fig. 1(b) · downsampler: keep x̂ where b = 1






Paper Fig. 15 · pooling alternatives: on par

> Keep the vectors where b = 1, drop the rest: 8 bytes become 3 chunks, and P travels along for dechunking. Fancier pooling does no better, so the encoder learns to pack each chunk into its kept vector.

# ④ Main network: an LM on learned tokens

RUNNING EXAMPLE · ④

p r o d u c t !

WHERE THE PARAMETERS GO · XL 1-STAGE (ESTIMATE)



**Encoder + decoder ≈ 50M (4%)**: 8 Mamba-2 layers × 6·1024²
**Main network ≈ 1.2B (96%)**: 24 Transformer layers × 12·2048²

SEQUENCE LENGTH

## 8,192 → ≈1,700

bytes → chunks; a BPE model sees 1,792 tokens for the same text

**Width change without a layer:** 1,024 dims + one shared learned vector of 1,024 → 2,048 dims in; keep the first 1,024 dims on the way out (SpaceByte's trick).

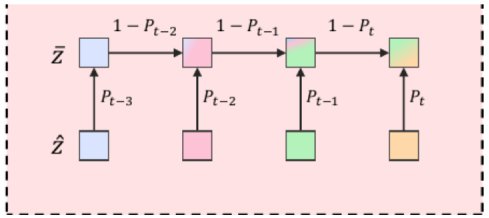
**Our example:** 3 chunk vectors in, 3 out:
$\hat{z}$ = [10, 20, 30]

> Sequences are shortest here, so this is where the parameters go. The main network is any standard language model, a Transformer in the paper's main runs, operating on learned “tokens” at roughly BPE length.

# ⑤a Smoothing module: forward pass



❓ **Question I had:** how does the dechunking layer turn 3 chunk vectors back into 8 byte vectors?



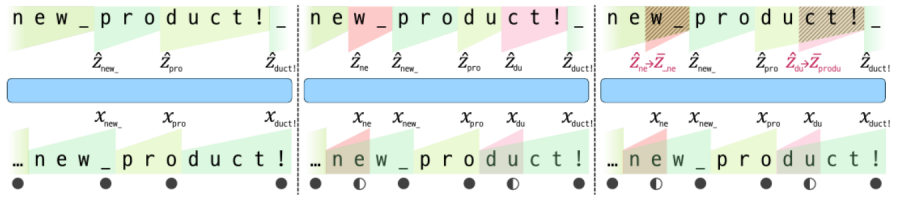

Paper Fig. 1(c) · smoothing module

$$\bar{z}_k = P_k \cdot \hat{z}_k + (1 - P_k) \cdot \bar{z}_{k-1}$$

"pro" P=1.0 → $\bar{z}_1 = 10$

"du" P=0.9 → 0.9·20 + 0.1·10 = 19

**"ct!" P=0.5 → 0.5·30 + 0.5·19 = 24.5**




Paper Fig. 4: (a) oracle boundaries · (b) no smoothing: "du" misaligns · (c) smoothing repairs the shaded regions

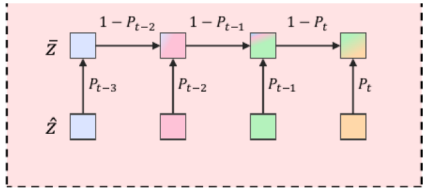
An EMA over chunks, weighted by confidence. The hesitant "ct!" becomes half "ct!", half "du", so the decoder still sees mostly "duct" context: a bad boundary does limited damage.

# ⑤b Smoothing module: backward pass



$$\partial \bar{z}_3 \,/\, \partial P_3 = \hat{z}_3 - \bar{z}_2 = 30 - 19 = 11$$

The gradient reaching $P_3$ is "new chunk minus old context": in the figure, the vertical arrow ($P_k$) against the horizontal arrow ($1 - P_k$).




Paper Fig. 1(c)

**New chunk "ct!"**
$\hat{z}_3 = 30$

⟵ $P_3$ ⟶

**Old context "du"**
$\bar{z}_2 = 19$

**New chunk helped the decoder** → $P_3$ rises, the boundary becomes confident.

**Old context was better** → $P_3$ falls; below 0.5 the cut disappears.

**Without smoothing**, P never enters the computation: no signal at all.

> Smoothing gives every boundary a gradient that asks: was starting a new chunk better than continuing the old one? It replaces the noisy Gumbel exploration that made earlier attempts unstable, and removing it is the most damaging ablation (Fig. 7).

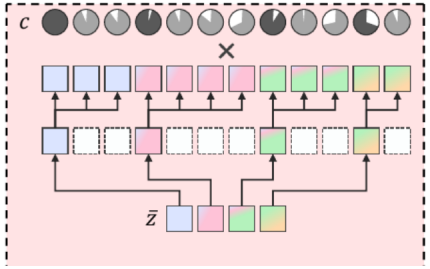
# ⑤c Upsampler: forward pass



p r o d u c t !

$c_t = p_t$ if $b_t = 1$, else $1 - p_t$ STE(c) = c + stopgrad(1 - c)
$z_t\tilde{}$ = $\bar{z}$ of the most recent chunk $out_t = STE(c_t) \cdot z_t\tilde{}$

| t · byte | 1 p | 2 r | 3 o | 4 d | 5 u | 6 c | 7 t | 8 ! |
|---|---|---|---|---|---|---|---|---|
| $b_t$ | 1 | 0 | 0 | 1 | 0 | 1 | 0 | 0 |
| repeat $z_t\tilde{}$ | 10 | 10 | 10 | 19 | 19 | 24.5 | 24.5 | 24.5 |
| $c_t$ | 1.0 | 0.8 | 0.9 | 0.9 | 0.7 | 0.5 | 0.95 | 0.6 |
| STE($c_t$) fwd | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 1 |
| $out_t$ | 10 | 10 | 10 | 19 | 19 | 24.5 | 24.5 | 24.5 |




Paper Fig. 1(d) · upsampler: repeat, × c

**Without the STE**, c·z̃ = 10, 8, 9, 17.1, 13.3, 12.3, 23.3, 14.7: magnitudes wobble with confidence, noise the decoder must undo.

Each chunk vector is copied to its bytes, **causally**: a byte only gets a chunk that started at or before it. The STE keeps the forward signal clean.

# ⑤d Upsampler: every byte gets a vote

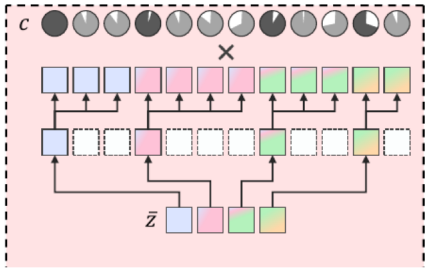
RUNNING EXAMPLE · ⑤

p r o d u c t !

$$\partial L/\partial p_t \propto (2b_t - 1) \cdot \langle \partial L/\partial \text{out}_t \, , \, z_t \rangle$$

the sign flips between boundary and non-boundary positions (main-horse's derivation)

| t · byte | 1 p | 2 r | 3 o | 4 d | 5 u | 6 c | 7 t | 8 ! |
|---|---|---|---|---|---|---|---|---|
| $b_t$ | 1 | 0 | 0 | 1 | 0 | 1 | 0 | 0 |
| $c_t$ is | p | 1-p | 1-p | p | 1-p | p | 1-p | 1-p |
| chunk helps here | p ↑ | p ↓ | p ↓ | p ↑ | p ↓ | p ↑ | p ↓ | p ↓ |
| chunk hurts here | p ↓ | p ↑ | p ↑ | p ↓ | p ↑ | p ↓ | p ↑ | p ↑ |




Paper Fig. 1(d) · the × c is where p gets its gradient

**Sharpening:** when a chunk helps, every p moves toward the extreme it already chose, 0 or 1. Confident routing is what makes F ≈ G for the ratio loss.

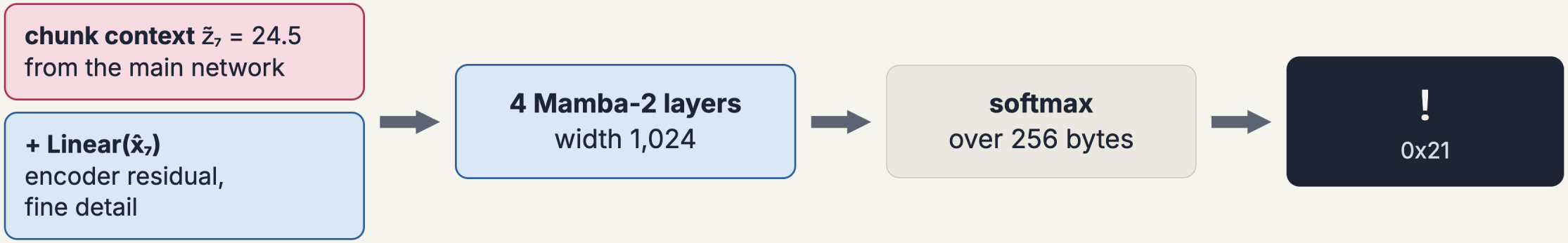
**Only route for non-boundaries:** bytes 2, 3, 5, 7, 8 never enter the smoothing module. This term is their only signal from prediction quality.

# ⑥ Decoder: spell it out byte by byte

RUNNING EXAMPLE · ⑥

| p | r | o | d | u | c | t | ! |
|---|---|---|---|---|---|---|---|

AT POSITION t = 7 (BYTE “t”): PREDICT THE NEXT BYTE




**Main network says** roughly “the word is product”. **Decoder** tracks the exact position inside the word.

**Analogy (§E.2):** the outer stages play the role of tokenizer + embedding table + LM head, but learned.

> The decoder combines coarse context from the main network with fine per-byte detail that never went through chunking. That residual projection is initialized near zero, so early training leans on the main network's path.

# Ratio loss: why it is needed

RUNNING EXAMPLE · LOSS

p r o d u c t !

❓ **Question I had:** how does the ratio loss work, and what is it for?

**Keep everything (F → 1):** no compression, the big main network runs on all 8,192 bytes. The prediction loss alone pulls this way.

**Keep almost nothing (F → 0):** the main network gets a handful of vectors and starves.

$$L_{ratio} = \frac{N}{N-1} \cdot [(N-1) \cdot F \cdot G + (1-F) \cdot (1-G)]$$

**F** = mean $b_t$: real fraction kept, hard, no gradient → the **sensor**
**G** = mean $p_t$: average probability, soft → the **knob**
**N** = target bytes per chunk

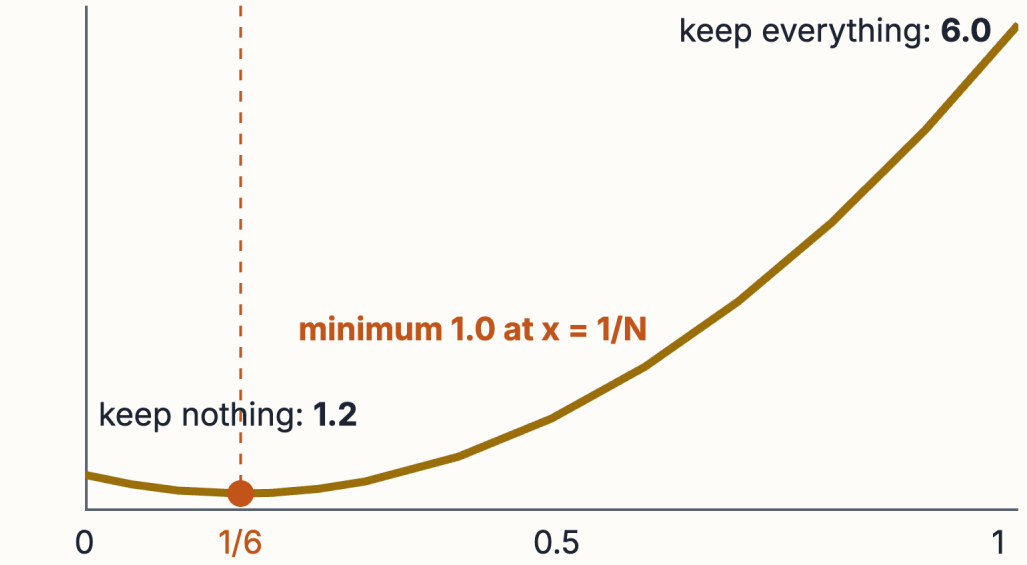
OUR EXAMPLE · TARGET N = 3

F = 3/8 = 0.375
G = 3.45/8 = 0.431
L ≈ 1.02 · F > 1/3

→ **push every $p_t$ down**

Borrowed from MoE load balancing. F counts real boundaries but has no gradient; G is its differentiable twin, so gradients flow through G. Total loss: $L = L_{AR} + \alpha \cdot \Sigma_s L_{ratio}$, with $\alpha = 0.03$.

# Ratio loss: a thermostat, not a hard constraint




$$\partial L/\partial G = N/(N-1)\cdot(N\cdot F - 1)$$

| F (N=6) | ∂L/∂G | effect |
|---|---|---|
| 0.30 | +0.96 | p ↓ fewer cuts |
| 1/6 | 0 | no push |
| 0.10 | −0.48 | p ↑ more cuts |

Same push on every position: it sets **how many** boundaries; prediction decides **where**.

**Soft guide:** 1-stage targeted 6 bytes/chunk, settled at **4.8**; 2-stage targeted 9, got **7.0**.

**Quirk:** with F ≠ G it can dip below 1 (≈0.99); confident routing makes F ≈ G.

**main-horse:** “compression is a contest”; over long training the main network's preference wins.

# Three signals remove the bad cut

**SMOOTHING (SLIDE 17)**

The decoder preferred “du” context (19) over the “ct!” fragment (30) → **$P_3$ ↓**

**CONFIDENCE c (SLIDE 19)**

At byte c the chunk vector hurt prediction, and c = $p_6$ there → **$p_6$ ↓**

**RATIO LOSS (SLIDE 21)**

F = 0.375 > 1/3: too many cuts → **every p ↓**




> No labels were used: the segmentation improves because a better split makes next-byte prediction easier. The post-training values (0.5 → 0.3) are illustrative; the mechanism is what the paper's equations imply.

# Making it train: signal propagation

### Norm balance

RMSNorm at the end of every sub-network (E, M, D). Otherwise the deep main network's growing residual norms drown out the encoder's fine detail.

### Two streams

A Linear projection only on the residual path, initialized near zero. Projections on the main path disturbed gradient flow and trained worse.

### LR per stage (eq. 11)

$\lambda^s \propto \sqrt{(\text{effective batch})} \cdot \sqrt{(D\text{^}S / D^s)}$. Outer stages, which control chunking, get higher learning rates and learn boundaries faster.

FROM GU'S “H-NETS: THE PAST” BLOG POST (PARAPHRASED)

About a dozen design choices had to interact correctly at once, and options kept flip-flopping as other parts changed; the deck of tried variants ran to hundreds of slides. Gumbel noise never helped, and a plain “sparsity loss” instead of the target ratio was less consistent.

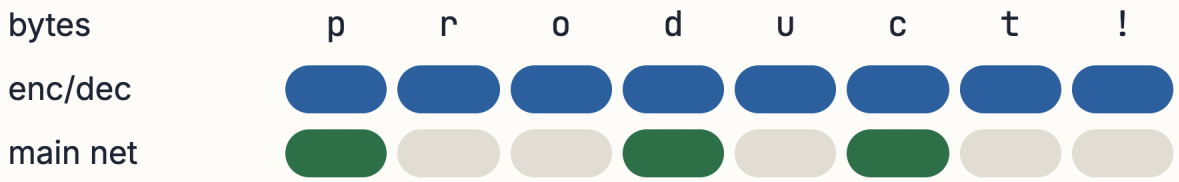
> These unglamorous details are what make end-to-end chunking stable beyond 1B parameters, where earlier dynamic-chunking attempts broke down.

# Going deeper, and how inference works

## 2-STAGE H-NET · XL · (3,3)-DC

- $E^0$ · M4 · width 1,024 · 8,192 bytes
- $E^1$ · T1M4 · width 1,536 · ≈ 3× shorter
- M · T27 · width 2,048 · ≈ 7 bytes per chunk
- $D^1$ · M4T1 · width 1,536
- $D^0$ · M4 · width 1,024 → next byte

## GENERATING “product!” · WHO STEPS WHEN




**Like speculative decoding:** a small model steps on every token, a big one every few. Gu argues H-Net builds this in.

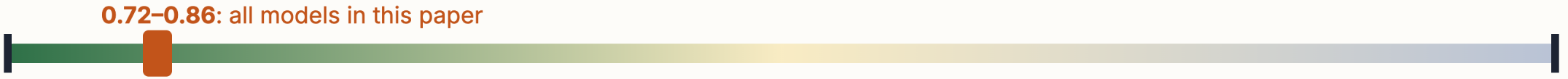
**Counterpoint (main-horse):** chunking follows how the encoder reads the input; in pretraining H-Net can't simply decide to “think more”.

> The main network can itself be an H-Net; two stages is the paper's best model. At inference the big network runs only at chunk boundaries (3 of 8 steps here), so compute per generated byte varies with content.

# Measuring quality: bits-per-byte

❓ **Question I had:** what is bits-per-byte, and why not just use the loss or perplexity?

THE BPB SCALE · LOWER IS BETTER




BPB = mean over bytes of
$-\log_2 p(\text{true next byte})$

**For a BPE model:** 3.4 bits/token ÷ 4.6 bytes/token ≈ **0.74 BPB**

**Why per byte?** Byte, BPE and learned-chunk models predict different units; bytes are shared.

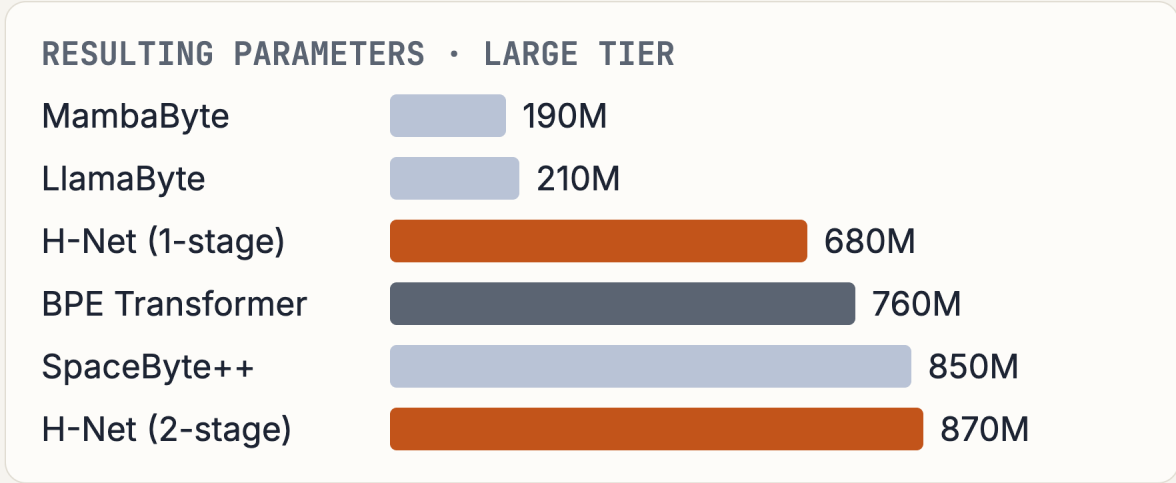
BPB is cross-entropy in bits, measured per byte. Gaps of 0.01 are meaningful. Caveat (§A): the standard conversion slightly overestimates a BPE model's BPB, a small bias in H-Net's favor; downstream results back the conclusions independently.

# A fair race: matched data, matched compute

❓ **Question I had:** what does matching “bytes per batch” and “FLOPs per byte” mean?

**Data:** 256 × 8,192 bytes ≈ **2.1M bytes/step** ≈ 256 × 1,792 BPE tokens × 4.6 bytes. Curves are plotted against training *bytes*.

**Compute:** every model is sized so one byte costs what it costs the BPE reference (GPT-3 Large 760M / XL 1.3B): ≈ **0.42 / 0.70 GFLOPs per byte**.




Same text, same compute, but very different sizes: isotropic byte models must be tiny, hierarchical models can afford more. FLOP-matched is not time-matched: the current H-Net code trains up to ~2× slower than an isotropic model.

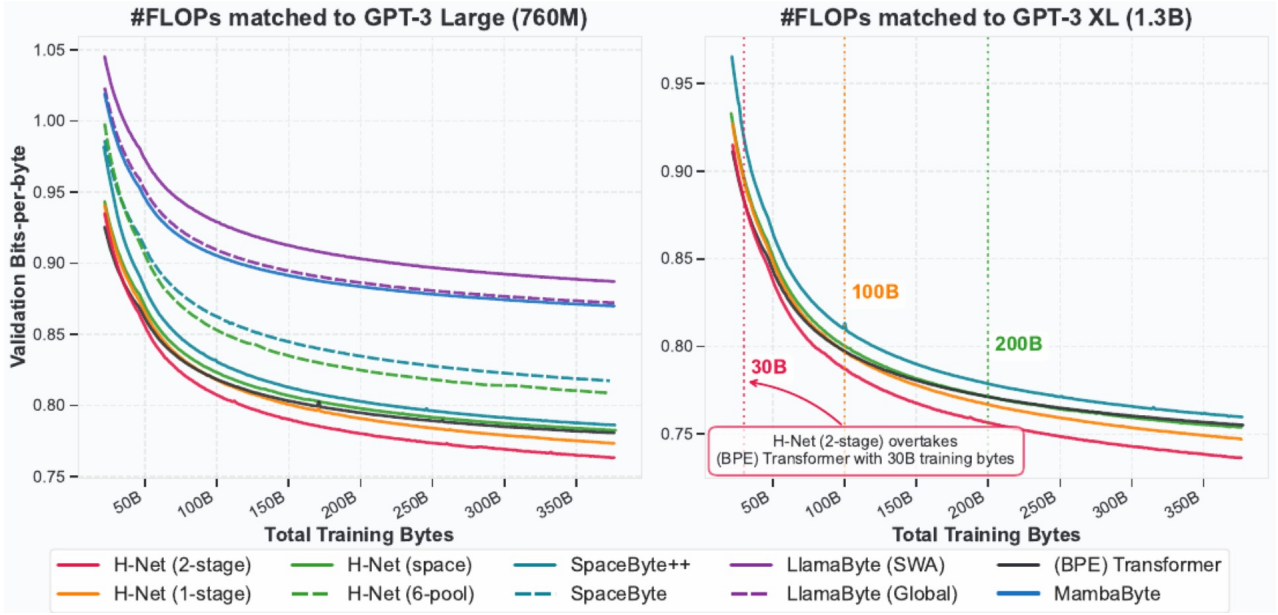
# The model ladder: one change per step

> ❓ **Question I had:** what are all these models, and how do they differ? (T = Transformer layer, M = Mamba-2 layer)

| Model | Chunks from | Architecture (Large) | What it isolates |
|---|---|---|---|
| BPE Transformer | GPT-2 tokenizer | T24 | the baseline to beat |
| LlamaByte · MambaByte | none (isotropic) | T16 (SWA) · M28 | naive byte models |
| SpaceByte | spaces (rule) | T8 + T16 + T8 | prior hierarchical model |
| SpaceByte++ | spaces (rule) | M4 + T28 + M4 | + Mamba outer layers |
| H-Net (space) | spaces (rule) | M4 + T28 + M4 | + training techniques |
| H-Net (pool) | every 6 bytes | M4 + T28 + M4 | content-blind chunking |
| H-Net (1-stage) | learned, N = 6 | M4 + T22 + M4 | learned vs. rules |
| H-Net (2-stage) | learned, 3 × 3 | M4+T1M4+T26+M4T1+M4 | nesting |

> Read top to bottom: each rung tests one idea. The cleanest comparison is the shaded block: pool vs. space vs. learned chunking inside an otherwise identical model.

# English: learned chunking beats BPE



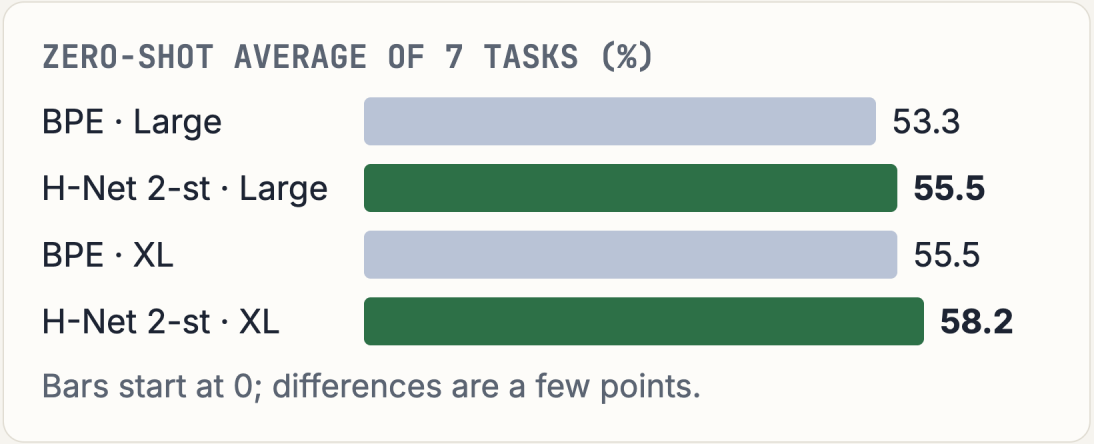

Paper Fig. 2 · left: Large (760M) · right: XL (1.3B), with crossover points

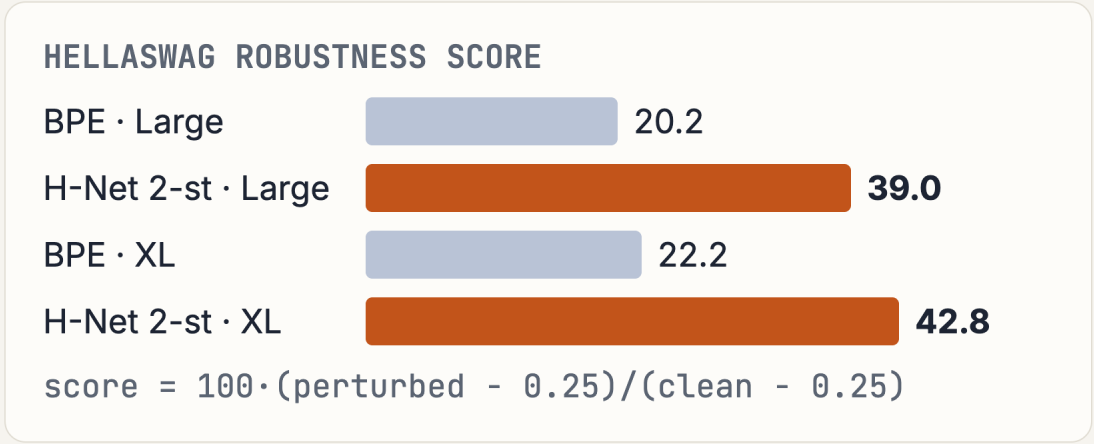
FINAL BPB (Large · XL)

| Model | L | XL |
|---|---|---|
| BPE Transf. | 0.756 | 0.730 |
| SpaceByte++ | 0.760 | 0.733 |
| H-Net pool | 0.780 | — |
| H-Net space | 0.755 | 0.726 |
| H-Net 1-st. | 0.755 | 0.728 |
| H-Net 2-st. | 0.743 | 0.715 |

> Isotropic byte models lose badly; architecture plus training tricks alone reach BPE. 2-stage is best and overtakes BPE after only ~30B bytes at XL, with a widening gap. Honest reading: 1-stage only ties the space rule on English, because it learns to cut at spaces anyway.

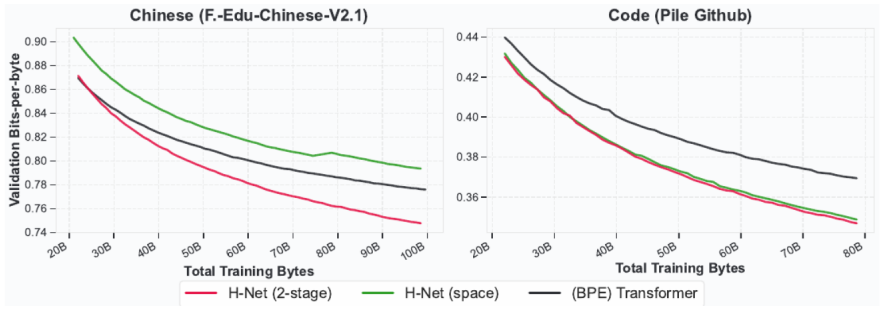
# Downstream accuracy and robustness to noise




> The Large 2-stage H-Net matches the XL Transformer on downstream average (55.5 = 55.5), though not on BPB (0.743 vs 0.730). Under typos and casing noise, BPE keeps about 1/5 of its above-chance ability and H-Net about 2/5; most of that comes from being byte-level at all (all byte models score ≈35–40).

# Chinese and code: where hand-made rules break




Paper Fig. 3 · XL scale · BPE uses the Llama-3 tokenizer

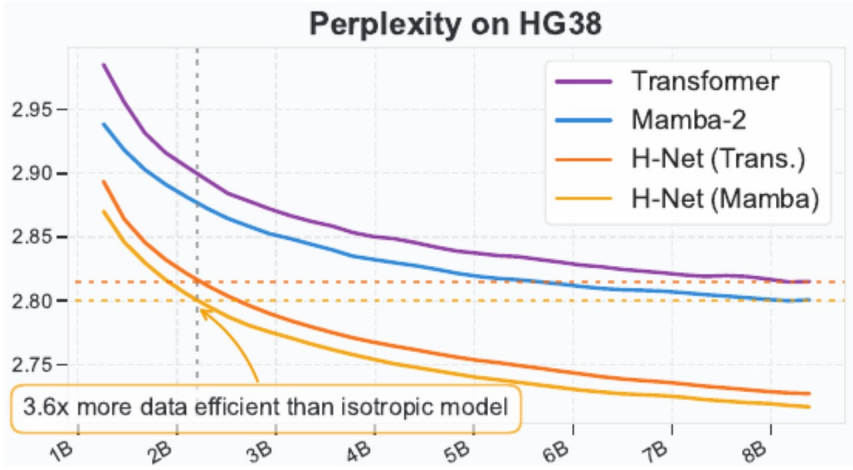
**Chinese:** no spaces, so the space rule loses to BPE; learned chunking wins clearly.

**Code:** lots of whitespace, so both H-Nets beat BPE by a wide margin and tie each other.

| Model | ZH BPB ↓ | XWinograd-zh ↑ | ZH bytes/chunk | Code BPB ↓ | Code bytes/chunk |
|---|---|---|---|---|---|
| BPE (Llama-3) | 0.740 | 59.9 | 3.62 | 0.338 | 3.58 |
| H-Net (space) | 0.748 | 62.9 | 3.38 | 0.316 | 7.97 |
| H-Net (2-stage) | 0.703 | 66.3 | 5.81 | 0.316 | 7.23 |

> Strongest evidence that learning *where* to cut matters. With one 3×3 target, H-Net settles at a sensible ratio per dataset; BPE and the space rule swing from 3.4 to 8 bytes.

# DNA: the bridge to our field



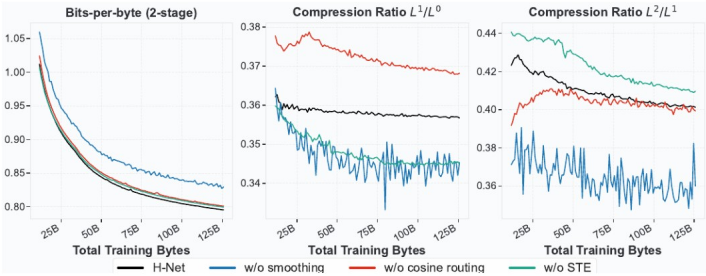

Paper Fig. 5 · HG38 human genome · stable phase

**3.6×** less data to reach the isotropic models' perplexity, for both main-network choices

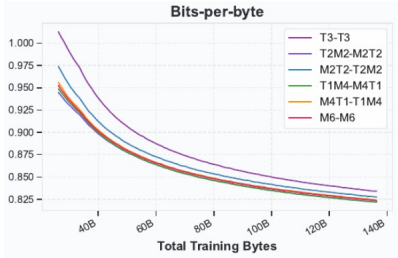
| Model | Params | Final PPL ↓ |
|---|---|---|
| Transformer T9 | 29M | 2.769 |
| Mamba-2 M10 | 33M | 2.757 |
| H-Net · main T15 | 64M | 2.705 |
| H-Net · main M15 | 66M | 2.697 |

A Mamba main network beats a Transformer on DNA, as known for isotropic models. Caveats: tiny models, H-Nets have ~2× the parameters, and 2-stage was never tried on DNA. An encoder with one attention layer (M3T1) helped on DNA but not on text.

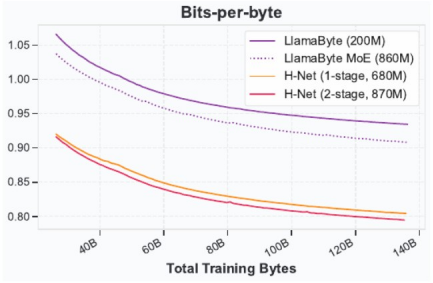
# What is essential? Four ablations



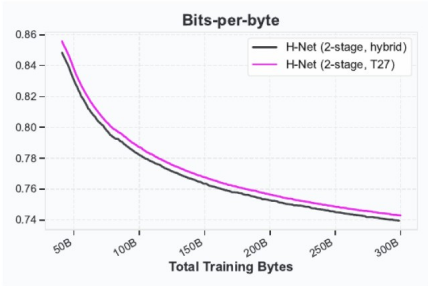

**Components (Fig. 7):** no smoothing hurts most; compression ratio swings.




**BPE inputs (Fig. 10):** Mamba still wins in E/D, so it's about compression, not bytes.

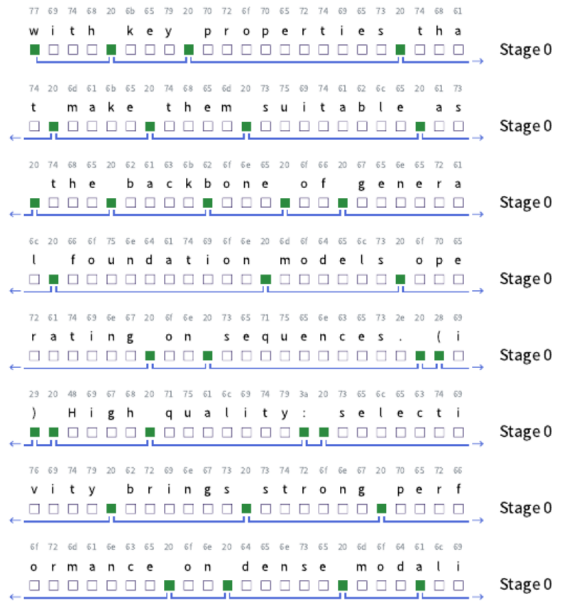



**vs. MoE (Fig. 14):** sparse byte Transformer, 860M params: far worse. Not just extra params.

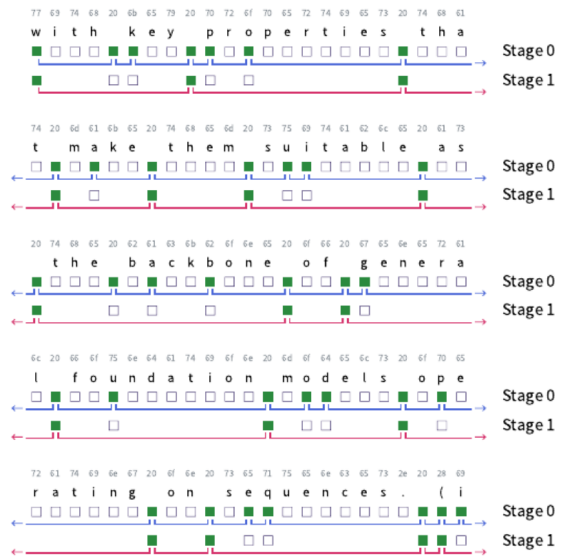



**Hybrid main net (Fig. 13):** 20 Mamba + 7 attention beats T27; LM lessons transfer.

# What did it learn? Boundaries without labels




(a) 1-stage · green = $b_t = 1$




(b) 2-stage · stage 0 and stage 1

**1-stage:** cuts mostly at spaces, rediscovering SpaceByte's rule on its own.

**2-stage:** stage 0 = spaces + word starts; stage 1 groups “the backbone”, “such as”, “(ii)”.

**Critique (main-horse):** without spaces, cuts land on each word's **first letter**, so the decoder must guess it without main-network help. Proteins have no delimiters either.

# Limitations and open questions

✓ **Strong design:** data- and compute-matched; consistent at two scales and across English, Chinese, code, DNA.

⚠ **Scale:** largest model is compute-matched to a 1.3B Transformer; 3B, 7B and beyond untested.

⚠ **Single runs:** no error bars; some benchmark gaps may be noise.

⚠ **No formal scaling laws:** "scales better" is read from curve slopes during the constant-LR phase.

⚠ **Wall-clock:** up to ~2× slower training; dynamic memory, uneven batches at inference.

⚠ **Knobs:** target N and $\alpha = 0.03$ are new hyperparameters; achieved ratio drifts from target.

> The authors flag most of these themselves in the Discussion. The BPB gaps are more trustworthy than the benchmark gaps, and the BPB conversion slightly favors H-Net.

# H-Net has already reached genomics (ICML 2026)

## DNAChunker

KAIST & Inocras · arXiv 2601.03019

**Objective:** masked LM (bidirectional)
**Backbone:** Caduceus + cross-attention
**Data:** human genome HG38
**Finding:** small chunks at promoters and exons, large in repeats; robust to shifts and mutations; reported to match 1.2B models with 172M parameters

## LDARNet

Ledneva & Kuznetsov · arXiv 2606.04552

**Objective:** masked LM, ≈120M params
**Backbone:** BiMamba-2 + local attention, ratio loss
**Data:** 27 NT + Genomic Benchmarks tasks
**Finding:** learned vs. fixed-grid cuts: up to +14 pts on histone tasks at equal FLOPs; cuts align with promoter motifs and splice junctions

## dnaHNet

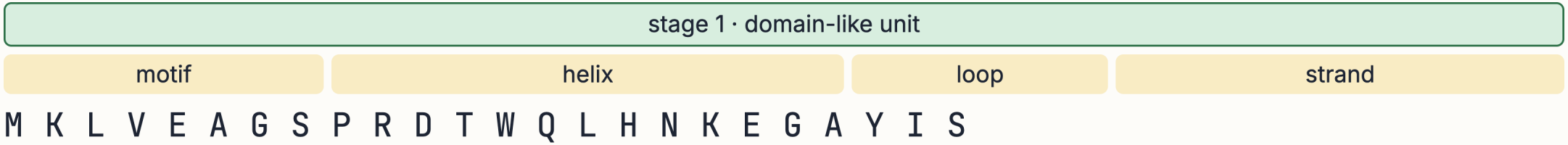
incl. Hwang & Gu · arXiv 2602.10603 · oral

**Objective:** autoregressive
**Data:** prokaryotic genomes
**Finding:** beats StripedHyena2 in scaling and efficiency; >3× faster inference than Transformers; zero-shot **protein variant fitness** and gene essentiality

> Two of three converted H-Net to bidirectional masked modeling, the ESM-style setting we use. dnaHNet already touches protein questions, but through DNA. All three report learned boundaries that line up with biology.

# Proteins: the open gap

A HYPOTHETICAL 2-STAGE HIERARCHY (ILLUSTRATIVE SEQUENCE)




**Objective:** autoregressive (ProGen-like) or masked (ESM-like)? DNA work leans masked.

**Target N:** a residue carries ≈4.3 bits vs. 2 for a base, so maybe a lower N.

**Encoder:** does DNA's M3T1 > M4 finding carry over?

**Boundaries:** no delimiters, so the first-letter issue applies.

**Evaluation:** compare cuts to Pfam, secondary structure, CATH; variant effects on ProteinGym.

**Efficiency:** proteins are short, so gains must come from abstraction, not saved compute.

> I found no protein adaptation of H-Net so far [verify before presenting]. Proteins have an obvious natural hierarchy and no good hand-made tokenizer: exactly where H-Net beat heuristics in Chinese and DNA.

TAKEAWAYS

# More learning, less processing

1. Learned chunking can replace the tokenizer at matched data and compute.
2. Nesting (2-stage) is where the clear wins appear.
3. Mamba belongs at high resolution because it compresses; attention belongs on meaningful units.
4. It shines where no good tokenizer exists: Chinese, code, DNA, and maybe proteins.

**Discussion: what would a “chunk” mean in a protein, and how would we verify it?**

FOR Q&A

# Backup slides

**B1** Running example master sheet

**B2** FLOP formulas and learning rates

**B3** Extras from Gu's blog posts

**B4** Training dynamics (main-horse)

**B5** Distilling Llama 3.2 3B into an H-Net

# Running example: all numbers in one place

| t | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 |
|---|---|---|---|---|---|---|---|---|
| byte · hex | p 70 | r 72 | o 6F | d 64 | u 75 | c 63 | t 74 | ! 21 |
| $\cos(q_t, k_{t-1})$ | -1.0 | 0.6 | 0.8 | -0.8 | 0.4 | 0.0 | 0.9 | 0.2 |
| $p_t$ | 1.0 | 0.2 | 0.1 | 0.9 | 0.3 | 0.5 | 0.05 | 0.4 |
| $b_t$ | 1 | 0 | 0 | 1 | 0 | 1 | 0 | 0 |
| chunk | pro | | | du | | ct! | | |
| $c_t$ | 1.0 | 0.8 | 0.9 | 0.9 | 0.7 | 0.5 | 0.95 | 0.6 |
| $\tilde{z}_t = \text{out}_t$ | 10 | 10 | 10 | 19 | 19 | 24.5 | 24.5 | 24.5 |

$P = [1.0, 0.9, 0.5] \cdot \hat{z} = [10, 20, 30]$
$\bar{z} = [10, 19, 24.5]$

$N = 3 \cdot F = 0.375 \cdot G = 0.431$
$L_{ratio} \approx 1.02 \cdot \partial L/\partial G = +0.19$

$\partial \bar{z}_3/\partial P_3 = 30 - 19 = 11$

no STE: $c \cdot \tilde{z} = 10, 8, 9, 17.1, 13.3, 12.3, 23.3, 14.7$

# FLOP counting and per-stage learning rates

### Attention (forward)

QKV: $2 \cdot 3 \cdot L \cdot d \cdot (h \cdot hd)$
logits: $2 \cdot L^2 \cdot (h \cdot hd)$
softmax: $3 \cdot h \cdot L^2$
score@V: $2 \cdot L^2 \cdot (h \cdot hd)$
out: $2 \cdot L \cdot (h \cdot hd) \cdot d$

### Mamba-2 (forward)

XZ: $2 \cdot L \cdot d \cdot (2 \cdot e \cdot d)$
BCΔt: $2 \cdot L \cdot d \cdot (2 \cdot N + h)$
SSD: $2 \cdot 3 \cdot L \cdot (e \cdot d) \cdot N$
conv: $2 \cdot L \cdot d \cdot w$ · gate: $5 \cdot L \cdot d$
out: $2 \cdot L \cdot d \cdot d$

### MLP · embeddings

gated MLP: $2 \cdot L \cdot 3 \cdot d \cdot f$
gating: $5 \cdot L \cdot d$
embed / head: $2 \cdot L \cdot V \cdot d$
backward = 2 × forward

$$\lambda^s = \sqrt{\left( N_GPT \cdot \Pi_{i=s..S} N_i \;/\; \Pi_{i=0..S} N_i \cdot D^S / D^s \right)},\quad N^S = 1$$

N_GPT = 4.6 bytes per GPT-2 token on FineWeb-Edu. Learning rate $\propto \sqrt{}$(effective batch) and $\propto 1/\sqrt{}$(width): outer, narrower, longer stages get larger multipliers.

Base learning rates: $6.25 \times 10^{-4}$ (Large), $5.0 \times 10^{-4}$ (XL), AdamW with warmup-stable-decay (10% warmup, 20% decay).

# Extras from the authors' blog posts

### BLT measures the wrong thing

BLT cuts on conditional entropy. Gu argues surprisal (−log p of the actual token) is better: if one word is expected but another appears, entropy stays low yet the model is very surprised.

### The filler-token litmus test

Insert meaningless filler tokens: the ideal model should not spend much more compute or memory. Attention blows up; SSMs still spend compute on every filler. Whether H-Net passes is open.

### Why fixed pooling fails for text

Language has no sampling rate; small spacing shifts change what a convolution or fixed pool sees. Gu's unreported S4-era U-Net was always worse on WikiText-103, yet worked for audio (SaShiMi).

### Origins and future

Gu first proposed information-based chunking; Hwang chose MoE-style routing instead. Early name: Hourglass Network with a "TickTok" layer. Future bets: audio/video, multimodal fusion, long context.

BACKUP B4 · MAIN-HORSE, “H-NET: INTUITIONS”

# Training dynamics seen by an independent replicator

**1 · Collapse**
compression ratio drops toward zero: the untrained main net is noise, so the model routes around it

→

**2 · Overshoot**
the main net becomes useful; the model sends it more bytes than it started with

→

**3 · Settle**
ratio-loss pressure wins; compression tapers to the target, all within ~500 steps

**Warning signs:** no quick hockey stick usually means a bad run. Ratio loss far above 1: too many bytes inward. Near its lower bound $N/(2(N-1))$: shutting the main net off.

**Compression is a contest:** mid-run ratio spikes were not always bad; suppressing them with a larger ratio weight made runs worse. Achieved ratio shifts with compute budget.

**Three meanings of “chunk”:** bytes the decoder predicts from one vector · vectors the main net receives · byte span the encoder groups.

**BPE is a zero-FLOP multiplier:** respect it; dynamic tokenizers open a new scaling axis, but their cost could still outweigh their gains.

BACKUP B5 · §E.2 & TABLE 7

# Distilling Llama 3.2 3B into a byte-level H-Net

**Setup:** frozen Llama 3.2 3B as main network; 4 Mamba-2 layers (width 1,536) as encoder and decoder; MLP adapters to width 3,072; 189B bytes of training.

**Extra losses:** BCE on boundaries (cut at every real token start) and L2 embedding matching (weight 0.02). The boundary loss is crucial.

| Model | Avg. acc (7 tasks) | MMLU 5-shot | Boundary acc. |
|---|---|---|---|
| Llama 3.2 3B (teacher) | 0.647 | 0.561 | — |
| Distilled H-Net (1-stage) | 0.617 | 0.519 | > 99.5% |
| Cutting only at spaces | — | — | < 95% |

**Tokenization bias:** Llama 3 splits “_distill” as [_dist, ill] but keeps “_distinct” whole. Whether i starts a token is unknown until later bytes arrive, so a causal router can never match BPE exactly.